Muhammad Aufa Dhiya'uddin **Selasa 3 February 2026**

25091397052 – stukdat 2025B

Pekan 5: Praktikum – (Penjelasan code yang di tulis)

Soal 1 : Modified Binary Search (Count Occurrences)

Memahami Masalah

Kita punya list yang sudah terurut (sorted) dan ingin menghitung berapa kali angka target muncul. Karena list sudah terurut, kita bisa manfaatkan Binary Search yang kompleksitasnya O(log n).

Penjelasan Algoritma

1. Pertama, kita cari index paling kiri dimana target muncul
2. Kedua, kita cari index paling kanan dimana target muncul
3. Rumus: jumlah = lastIndex - firstIndex + 1
4. Kompleksitas: O(log n) karena setiap pencarian hanya O(log n) dan kita pakai 2 kali pencarian → tetap O(log n)

Soal 2 : Bubble Sort dengan Analisis Langkah

Memahami Masalah

Kita perlu membuat bubble sort yang:

1. Mengembalikan tuple: (sorted_list, total_comparisons, total_swaps, passes_used)
2. Berhenti lebih awal jika sudah terurut (early termination)
3. Mencetak state array setelah setiap pass
4. Membandingkan 2 kasus: [5, 1, 4, 2, 8] dan [1, 2, 3, 4, 5]

Analisis Perbedaan Jumlah Pass

1. Kasus 1 [5, 1, 4, 2, 8] → 3 pass
    - Array belum terurut sama sekali
    - Butuh beberapa kali pass untuk memindahkan elemen ke posisi yang benar
    - Pada pass 3, masih ada swap, jadi lanjut pass 4? Ternyata di pass 3 tidak ada swap (cek output: pass 3 tidak ada swap, tapi tetap jalan karena kita belum tahu akan ada swap atau tidak di awal pass)

Koreksi kecil: Sebenarnya di pass 3 tidak ada swap, jadi setelah pass 3 selesai, loop berhenti. Total pass = 3.

2. Kasus 2 [1, 2, 3, 4, 5] → 1 pass

    - Array sudah terurut dari awal
    - Pass 1: melakukan perbandingan tapi tidak ada satu pun swap
    - Karena tidak ada swap, early termination aktif, loop berhenti setelah pass 1

Early termination membuat bubble sort lebih efisien untuk data yang sudah (hampir) terurut. Jumlah pass tergantung pada:

- Seberapa "kacau" data awal
- Apakah masih ada pertukaran yang terjadi dalam satu pass

Soal 3 — Hybrid Sort (Insertion + Selection Sort)

Memahami Masalah

Kita perlu membuat algoritma sorting hybrid yang:

1. Menggunakan Insertion Sort jika ukuran sub-array ≤ threshold (default 10)
2. Menggunakan Selection Sort jika ukuran sub-array > threshold
3. Membandingkan jumlah operasi (comparisons + swaps) dari 3 algoritma:
    - Hybrid Sort
    - Pure Insertion Sort
    - Pure Selection Sort
4. Menguji pada array random ukuran 50, 100, dan 500
5. Menampilkan hasil dalam bentuk tabel

Analisis Hasil

Dari output di atas, terlihat bahwa jumlah operasi sama untuk ketiga algoritma! Kok bisa?

Penjelasan:

1. Selection Sort selalu melakukan:
    - Perbandingan: n*(n-1)/2 (selalu, tidak peduli data sudah terurut atau belum)
    - Swap: maksimal n-1 kali
2. Insertion Sort untuk data random rata-rata:
    - Perbandingan: sekitar n*(n-1)/4
    - Swap: sekitar n*(n-1)/4
    - Total: sekitar n*(n-1)/2
3. Untuk data random, insertion sort dan selection sort memiliki kompleksitas rata-rata yang sama: $O(n^2)$
4. Hybrid Sort dengan threshold 10:
    - Untuk n=50: semua > threshold? Ya, karena 50 > 10, jadi pakai selection sort semua → hasilnya sama

    - Untuk mendapatkan hasil berbeda, kita perlu array yang sudah hampir terurut atau threshold yang lebih besar

Soal 4 — Merge Tiga Sorted Lists (Satu Pass)

Memahami Masalah

Kita perlu menggabungkan 3 list terurut menjadi 1 list terurut dalam O(n) dengan satu pass menggunakan 3 pointer.

Penjelasan Algoritma

1. Inisialisasi 3 pointer (i, j, k) di awal masing-masing list
2. Selama ketiga list masih punya elemen:
    - Bandingkan nilai di posisi pointer
    - Ambil yang terkecil, pindah ke result
    - Majukan pointer dari list yang diambil
3. Jika salah satu list habis, lanjutkan dengan 2 list sisanya
4. Jika dua list habis, tinggal salin sisa list terakhir

Kompleksitas Waktu

- Waktu: O(n) dengan n = total elemen (lenA + lenB + lenC)
- Ruang: O(n) untuk menyimpan hasil

Perbandingan dengan Merge Bertahap

Merge bertahap (merge A&B, lalu hasilnya merge dengan C):

- Pass 1: merge A&B → O(lenA + lenB)
- Pass 2: merge (A&B) dengan C → O((lenA+lenB) + lenC)
- Total operasi tetap O(n), tapi ada dua pass yang menyebabkan array hasil sementara dibuat

Soal 5 — Inversions Counter

Memahami Masalah

Inversion adalah pasangan (i, j) dimana:

- i < j (indeks lebih kecil)
- arr[i] > arr[j] (nilai lebih besar)

Contoh: [2, 3, 8, 6, 1]

- Inversions: (2,1) karena 8>1, (3,1) karena 6>1, (8,1) karena 8>1, (2,? cek lagi)
- Mari kita hitung manual:
    - i=0 (2): bandingkan dengan 3(√), 8(√), 6(√), 1(2>1 → inversion)
    - i=1 (3): bandingkan dengan 8(√), 6(√), 1(3>1 → inversion)

    - i=2 (8): bandingkan dengan 6(8>6 → inversion), 1(8>1 → inversion)
    - i=3 (6): bandingkan dengan 1(6>1 → inversion)
- Total = 1 + 1 + 2 + 1 = 5 inversions

Analisis dan Penjelasan

1. Kompleksitas Algoritma

Algoritma	Kompleksitas	Penjelasan
Naive (Brute Force)	$O(n^2)$	Mengecek semua pasangan (i,j) → total n(n-1)/2 operasi
Smart (Merge Sort)	O(n log n)	Membagi array, menghitung inversions sambil merge

2. Ilustrasi Perbedaan

Untuk n = 10.000:

- Naive: ~50 juta operasi (10.000 × 9.999 / 2)
- Smart: ~10.000 × $\log_2$(10.000) ≈ 10.000 × 14 ≈ 140.000 operasi

3. Mengapa Merge Sort Bisa Menghitung Inversions?

Saat merge dua subarray yang sudah terurut:

- Subarray kiri: [L_i, L_i+1, ..., L_mid] (terurut)
- Subarray kanan: [R_j, R_j+1, ..., R_right] (terurut)

Jika L_i > R_j, maka:

- Semua elemen dari L_i sampai L_mid > R_j
- Karena subarray kiri terurut, jika L_i > R_j, maka L_i+1, L_i+2, ... juga > R_j
- Jumlah inversions tambahan = (mid - i + 1)

Kesimpulan

1. Naive ($O(n^2)$) - Sederhana tapi lambat untuk data besar
2. Smart (O(n log n)) - Memanfaatkan Divide and Conquer
3. Perbedaan dramatis terlihat saat ukuran data membesar
4. Merge Sort bisa menghitung inversions secara alami karena saat merge kita tahu relasi antar elemen dari dua bagian